14th Aug 2020
Friday

Data Error Detection & Correction → Method: Parity → Even / Odd

- DLL responsible for this

Error
- 1-Bit error (single " ")
- Burst error

1	1	0	1	0	1	1	1

↓ Even parity bit

No. of 1's in the word = odd

⇓

{ If even, then parity bit = 0 }

⇒ Even parity-bit of $(27)_{10}$ = ?

$(27)_{10} = (0011011)_2$

Even parity-bit = 0

↓

0 0011011

⇒ Generate even parity-bit of "Data".

Data → 68 97 117 97

= 1000100 1100001 1110100 1100001

*** Write this first in exam

Horizontal / even P.B.

D	0	1000100
A	1	1100001
T	0	1110100
A	1	1100001
	0	0110000

← Vertical even P.B. (used in synchronisation)

Hor. + Vert.

⇒ "Science"

Science → 83 99 105 101 110 99 101

S	0	1	0	1	0	0	1	1
c	0	1	1	0	0	0	1	1
i	0	1	1	0	1	0	0	1
e	0	1	1	0	0	1	0	1
n	1	1	1	0	1	1	1	0
c	0	1	1	0	0	0	1	1
e	0	1	1	0	0	1	0	1

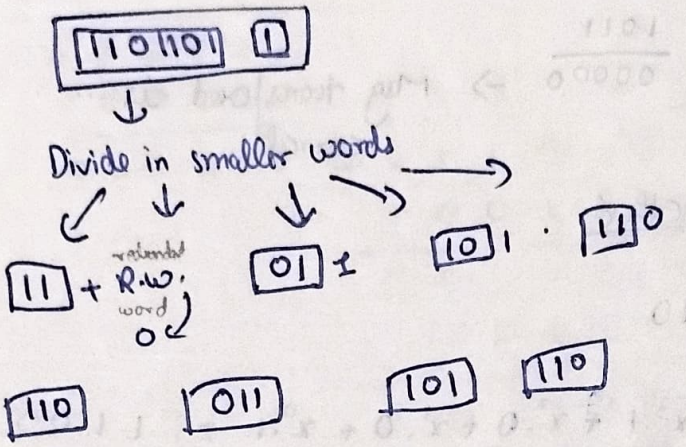
→ 10101100

Monday
17 Aug. 2026

Hamming Distance ← No. of differences b/w corresponding bits

← Shared along with message

Codeword = Dataword + redundant bits



D.W.	C.W.
00	000
01	011
10	101
11	110

$d(110, 101)$ = No. of bits changed

$= \begin{matrix} 1 \\ 1 \end{matrix} \begin{matrix} 1 & 0 \\ 0 & 1 \end{matrix}$

$= 2$

$d(11110, 010) = \begin{matrix} 1 \\ 0 \end{matrix} \begin{matrix} 1 \\ 0 \end{matrix} \begin{matrix} 1 \\ 0 \end{matrix} \begin{matrix} 1 & 0 \\ 1 & 0 \end{matrix}$

$= 3$

Cyclic Redundancy Check (CRC) for error detection

↳ Take D.W. & add redundant bits similar to Hamming distance.

m = no. of bits in msg (= 1001)

r = redundant bits (= Divisor − 1 = 3) (000)

Code word (C) = m + r = dividend

let divisor = 1011

XOR

```
      1010
1011 | 1001000
       1011
       -----
        0010000
          1011
          ----
          00110
```

Odd → 1
Even → 0

C.W. = Msg + rem.
= 1001 + 110 = D.W. 1001 | R.B. 110

Now, 1011) 1001110

1011

00101 1

1011

00000 → Msg transfered is correct

Cross-checking at receiver's end.

#Polynomial method

Msg = 1010101010

Divisor = $x^4 + x^3 + 1$

↳ $x^4.1 + x^3.1 + x^2.0 + x^1.0 + x^0.1$ = 11001

Find red. bits & make C.W.

R.B. = Divisor - 1 bits = 4-bits (0000)

```
                11000100
        ┌──────────────────
11001 ) 10 10 10 10 10 0000
        11 00 1↓
        ─────────
        01 1 0 00
          11 0 01
          ────────
          00 0 0 1 1 0 1 0
                11 0 0 1
                ─────────
                0 0 0 1 1 0 0 0
                      1 1 0 0 1
                      ─────────
                      0 0 0 0 1 0
```

Hence, C.W. = 1010101010 0010

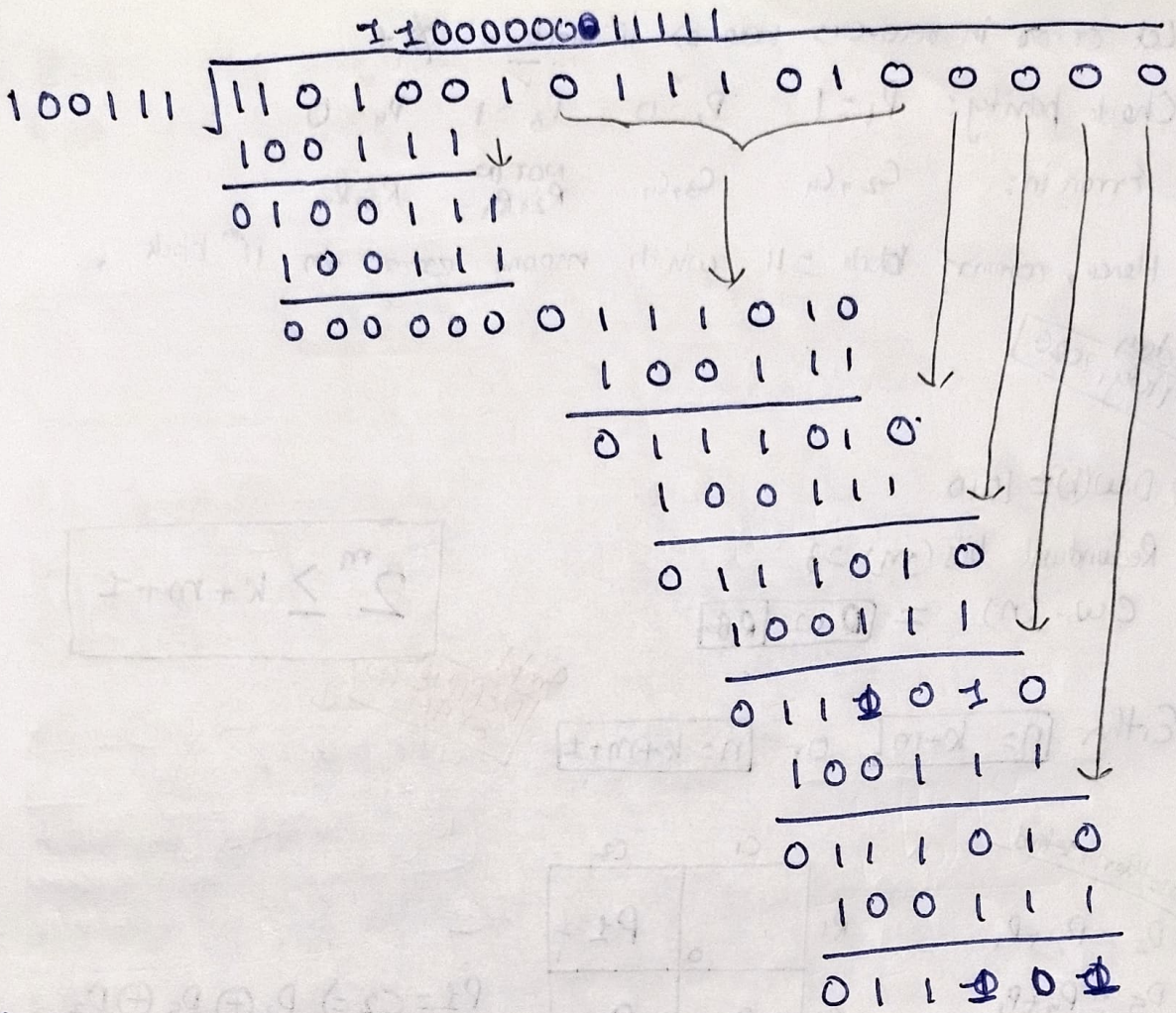
⇒ Msg = $x^{12} + x^{11} + x^9 + x^6 + x^4 + x^3 + x^2 + 1$

= 1101001011101

Div = $x^5 + x^2 + x^1 + 1$

= 100111

R.B. = 5-bits



C.W. =

1	1	0	1	0	0	1	0	1	1	1	0	1	1	1	0	1

Tuesday
18 Aug, 2026

Error Detection & Correction

⇒ **Hamming Code**

We have msg with some no. of bits
[= 00 11 000 1110 (11-bits) {let}]

Parity = redundant bits = 4-bits {+1 later = 16-bits}

$P_1\ P_2\ P_3\ P_4$
$2^0\ 2^1\ 2^2\ 2^3$

C.W. = $2^n - 1$

Even parity of whole matrix

	C_1	C_2	C_3	C_4
R_1	0_0	$P_1=0_1$	$P_2=1_2$	0_3
R_2	$P_3=1_4$	0_5	1_6	1_7
R_3	$P_4=1_8$	0_9	0_{10}	0_{11}
R_4	1_{12}	1_{13}	1_{14}	0_{15}

$P_1 = C_2 + C_4 = 0$ {already even 1s}
$P_2 = C_3 + C_4 = 1$
$P_3 = R_2 + R_4 = 1$
$P_4 = R_3 + R_4 = 1$

C.W. = 16-bits
0010 1011 1000 1110

Remove this when 15-bits are asked.

Let error in receiver's side ⇒ 11th block = ~~0~~ 1

⇒ Check parity: $P_1 = 1$ ✗ $P_2 = 0$ ✗ $P_3 = 1$ ✓ $P_4 = 0$ ✗

Error in: C_2, C_4 | C_3, C_4 | NOT IN R_2, R_4 | R_3, R_4

Hence, common block = 11 which means error in 11th block

Friday
21 Aug, 2026

⇒ D.W(k) = 1010

Redundant bits (m) = 3

C.W. (n) = | D.W. | R.B. |

$$2^m \geq k + m + 1$$

gmb! DEFINE IN EXAM

Either $n = k+m$, or $n = k+m+1$

Little Endian → Small value first

Big → Large value first

Another Method

$D_3 = P_2 + P_1$

$D_5 = P_4 + P_1$

$D_6 = P_4 + P_2$

$D_7 = P_4 + P_2 + P_1$

$P_1 = P_3 \oplus D_5 \oplus D_7$

$P_2 = P_3 \oplus D_6 \oplus D_7$

$P_4 = D_5 \oplus D_6 \oplus D_7$

	C1	C2
R1	0	$P_1 = 1$ (1)
R2	$P_2 = 0$ (2)	D_3 1 (3)
R3	$P_4 = 1$ (4)	D_5 0 (5)
R4	D_6 1 (6)	D_7 0 (7)

$P_1 = C_2 \Rightarrow D_3 \oplus D_5 \oplus D_7$

$= 1$

$P_2 = R_2 \& R_4 \Rightarrow D_3 \oplus D_6 \oplus D_7$

$= 0$

$P_4 = R_3 \& R_4 \Rightarrow D_5 \oplus D_6 \oplus D_7$

$= 1$

C.W. = 10110101 sent

Suppose receiver received = 1011000

Acc. to table ⇒ $P_1 = 1$ NO! = 0

$P_2 = 1$ YES! = 1

$P_4 = 0$ YES! = 1 (Error occured)

110 = 6th position error

⇒ Flip the no. at position 6.

⇒ Assume that 12-bit Hamming code consists of 8-bit data & 4-bit parity bits. D.w. + P.B. = | 1 1 0 x 0 1 0 1 | y 0 1 0 | (D.w. | P.B.; $P_8 P_4 P_2 P_1$)

→ NOT ACTUAL REPRESENTATION

Little Endian

D_{12}	D_{11}	D_{10}	D_9	P_8	D_7	D_6	D_5	P_4	D_3	P_2	P_1
1	1	0	x	y	0	1	0	0	1	1	0

$P_1 = D_3 \oplus D_5 \oplus D_7 \oplus D_9 \oplus D_{11}$

$0 = 1 \quad 0 \quad 0 \quad x \quad 1$

⇒ $\boxed{x = 0}$

$P_2 = D_3 \oplus D_6 \oplus D_7 \oplus D_{10} \oplus D_{11}$

$P_3 = D_9 \oplus D_{10} \oplus D_{11} \oplus D_{12}$

$y = x_0 \quad 0 \quad 1 \quad 1$

⇒ $\boxed{y = 0}$

⇒ Given D.w. $(97)_{10}$ 'a'. Find redun. bits & c.w. generated by hamming code.

⇒ Suppose data sent by ps is $(97)_{10}$ 'a' & received is $(98)_{10}$ 'b'. Find location at which 1 is changed using hamming code.

① → Use little endian

a = 1100001 P.B. = 4

D_{11}	D_{10}	D_9	P_8	D_7	D_6	D_5	P_4	D_3	P_2	P_1
1	1	0	0	0	0	0	0	1	1	0

$P_1 = D_3 \oplus D_5 \oplus D_7 \oplus D_9 \oplus D_{11} = 0$

$P_2 = D_3 \oplus D_6 \oplus D_7 \oplus D_{10} \oplus D_{11}$ 1

$P_4 = D_5 \oplus D_6 \oplus D_7 = 0$

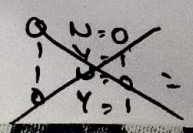
$P_8 = D_9 \oplus D_{10} \oplus D_{11} = 0$

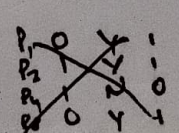
c.w. = 110000001100

(ii)

b = 1100010 ⇒

D_{11}	D_{10}	D_9	P_8	D_7	D_6	D_5	P_4	D_3	P_2	P_1
1	1	0	0	0	0	1	1	0	1	0





} Gives incorrect position but detects errors

Disadv: Works incorrectly for 2-bit error or more than 2-bit e.

(11)

a = 1100001

b = 1100010

Monday 24 Aug, 2020

Checksum

- Both decimal & binary
- If msg = (7,11,12,0,6) ⇒ we send (7,11,12,0,6,36) (4-bits = sum)
  - 36 > 4-bits (Problem)
- Receiver check the sum & compare
- we send negative (complement) of sum, c/s checksum
  - (7,11,12,0,6,-36)
- Receiver adds all, if sum = 0 ⇒ no error.

Sender side

- 36 = 100100
  - = 10 + 0100
  - = 0110 = 6 (wrapped sum)
- -6 = 1001 = 9 (checksum)

Receiver side

- (7,11,12,0,6,9)
- sum = 45 = 101101
  - = 10 + 1101
  - = 1111
- complement = 0000
  - = 0 (Hence, no error)

Solution

If 21 = 10101

Bits wrapping ↓ x: 4-bits

1 + 0101

= 0110 (= 6)

1001 = -6 (complement)

{ = 9 unsigned }

- Ex: Forouzan (16-bits representation)
  - = fo ro uz an
  - ↳ (complement taken will be 15's

⇒ Checksum on "Analysis"

ASCII → write this also in exam

Sender

An → 65 110 → 4 1 6 E
al → 97 108 → 6 1 6 C
ys → 121 115 → 7 9 7 3
is → 105 115 → 6 9 7 3

1 8 5 C 0
+ 1
8 5 C 1

Receiver

8 5 C 1
7 A 3 E
F F F F

Compl. = 0000

Hence, no error

Checksum = 15th complement = 15 - (85C1)
= 7A3E

⇒ ChatGPT

Sender

ASCII

OC → 0, 67 → 0043
hq → 104, 97 → 6861
tq → 116, 71 → 7447
PT → 80, 84 → 5054

1 2 D 3 F
+ 1
2 D 4 0

Receiver

2 D 4 0
D 2 B F
F F F F

Compl. = 0000

Hence, no error

Checksum = FFFF - 2D40
= D2BF

Unit → 2 → the Data link layer.

↳ Two sublayers.

Broadcast link → point to point link

framing & flow & error control

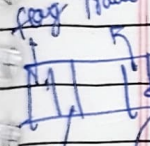
Data link control → deals with procedures for communication b/w 2 adjacent nodes → Node to Node communication no matter whether the line is dedicated or broadcast.

DLL: Framing.

→ Comm^n at data link layer is node-to-node.
→ DLL is located b/w Physical & Network layers.
→ First service by DLL is framing. frames.
→ DLL at each node needs encapsulate the datagram (packet recieved from Network layer) in a frame before sending it to next node.
→ DLL adds header & trailer to the datagram.

Frame: Types:

→ fixed length frames
• Sender don't have to specify start & end of frame.
• the size itself can be used as a delimiter.
• generally used by ATM (Asynchronous Mode)
→ Variable length frames
• Senders have to specify start and end of frame.
• Used in ethernet LAN.



→ Character oriented approach → divide every character in frames.
• Bit oriented approach
→ If flag appears in data then byte stuffing is used.
Escape, flag.
Technique mainly used to send ASCII messages.

Bit oriented approach → similar to byte oriented approach
→ Data section of frame is sequence of bits to be interpreted by upper layer as text, graphic, audio, video and so on.
→ flag to detect frame boundaries.
→ Most protocols use a special 8 bit pattern flag, as the delimiter to define beg & end of frame.
→ If flag appears in data.

Flow & error control.

ACK, ARQ At DLL, we deal with 4 entities:
↖ error control is implemented by

Monday
7 Sept, 2026

Protocols

- Noiseless channel
  - Simplest
  - Stop & Wait
- Noisy channel
  - Stop & wait ARQ (Automatic Repeat Request)
    - special case of
  - Go back-N ARQ
  - Selective Repeat ARQ
    - size of S & R window at most one-half of 2^m

Simplest

```
while(true){
    WaitForEvent();
    if(Event(Req To Send)){
        GetData();
        MakeFrame();
        SendFrame();
    }
}
```

Sender

```
// Repeat forever
// Sleep until an event occurs
// There is a packet to send
if Event(ArrivalNotification)){
    ReceiveFrame();
    ExtractData();
    DeliverData();
}
```

Receiver

↳ A random guy commonly known as FLY